

Weekly Assignment 5: React-SpringBoot Integration (Real-Time Chat Application)

This assignment details the connection architecture and configuration details for integrating the React Frontend with the Spring Boot Backend in the premium Real-Time Chat Application.

1. General Connection Information

The application utilizes two main communication channels to support user access and instant message synchronization:

Component	Port / Host	Description / Protocol
Frontend Client	http://localhost:5173	React Single Page Application (SPA) running via Vite.
Backend REST API	http://localhost:8080	Spring Boot REST endpoints, handling authorization, user management, and room listings.
WebSocket Protocol	ws://localhost:8080/ws	Live STOMP message broker endpoints for room streams and active presence tracking.

2. Spring Boot Backend CORS Configuration

CORS settings are configured globally inside the security context wrapper ([WebSecurityConfig.java](#)) on the Spring Boot backend to allow incoming cross-origin connections with credentials.

WebSecurityConfig.java (CORS configuration bean)

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOriginPatterns(List.of("*"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
```

```

        configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "Cache-Control"));
        configuration.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }

```

This source is injected into the filter chain:

```

http.cors(cors -> cors.configurationSource(corsConfigurationSource()))

```

3. Frontend REST Integration (Axios Wrapper & Token Interceptor)

REST actions route through an initialized Axios wrapper in `src/services/api.js` which automatically injects authentication tokens by parsing local user records.

api.js (Axios wrapper with interceptor)

```

import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:8080/api',
  headers: {
    'Content-Type': 'application/json'
  }
});

// Request interceptor extracting JWT dynamically from local state
api.interceptors.request.use(
  (config) => {
    const storedUser = localStorage.getItem('user');
    if (storedUser) {
      const { token } = JSON.parse(storedUser);
      if (token) {
        config.headers['Authorization'] = `Bearer ${token}`;
      }
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

export default api;

```

4. Auth State Context & Backend Sign Out

Authentication state is kept in `AuthContext.jsx`. The logout function calls the backend logout endpoint before clearing state.

AuthContext.jsx (Login & Logout handlers)

```
const login = (userData) => {
  setUser(userData);
  localStorage.setItem('user', JSON.stringify(userData));
};

const logout = async () => {
  try {
    await api.post('/auth/logout');
  } catch (err) {
    console.error('Logout error on backend', err);
  } finally {
    setUser(null);
    localStorage.removeItem('user');
  }
};
```

5. WebSocket STOMP Message Broker Integration

Instant messaging and user status updates are handled over a STOMP broker.

5.1 Backend Handshake Validation (`WebSocketConfig.java` interceptor)

An inbound channel interceptor extracts the Bearer JWT token from the STOMP connection frame to authenticate the connection:

```
@Configuration
@EnableWebSocketMessageBroker
@Order(Ordered.HIGHEST_PRECEDENCE + 99)
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
  @Autowired
  private JwtUtils jwtUtils;
  @Autowired
  private UserDetailsServiceImpl userDetailsService;
  @Override
```

```

    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*").withSockJS();
        registry.addEndpoint("/ws").setAllowedOriginPatterns("*");
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/topic", "/queue", "/user");
        registry.setApplicationDestinationPrefixes("/app");
        registry.setUserDestinationPrefix("/user");
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(new ChannelInterceptor() {
            @Override
            public Message<?> preSend(Message<?> message, MessageChannel channel) {
                StompHeaderAccessor accessor = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
                if (accessor != null && StompCommand.CONNECT.equals(accessor.getCommand())) {
                    String authHeader = accessor.getFirstNativeHeader("Authorization");
                    if (authHeader != null && authHeader.startsWith("Bearer ")) {
                        String token = authHeader.substring(7);
                        if (jwtUtils.validateJwtToken(token)) {
                            String username = jwtUtils.getUserNameFromJwtToken(token);
                            UserDetails userDetails = userDetailsService.loadUserByUsername(username);
                            UsernamePasswordAuthenticationToken authentication =
                                new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
                            accessor.setUser(authentication);
                        }
                    }
                }
                return message;
            }
        });
    }
}

```

5.2 Frontend Client Connection & Messaging Excerpt

Using `@stomp/stompjs`, the client connects and registers room subscriptions:

```

// Connection configuration
const stompClient = new Client({
  brokerURL: 'ws://localhost:8080/ws',
  connectHeaders: { Authorization: `Bearer ${user.token}` }
});

// Subscription for chat messages
stompClient.subscribe(`/topic/room/${roomId}`, (message) => {

```

```
const incomingMessage = JSON.parse(message.body);
setMessages(prev => [...prev, incomingMessage]);
});

// Publishing a chat message
stompClient.publish({
  destination: `/app/chat/${roomId}`,
  body: JSON.stringify({ content: text, sender: user.username })
});
```